

META-HEURÍSTICA ALNS PARA O PROBLEMA DE SEQUENCIAMENTO DE LOTES EM MÁQUINAS PARALELAS: APLICAÇÃO EM UMA LAVANDERIA INDUSTRIAL

Vicente Zawadzki Trindade^{1*}

*Autor correspondente

¹Rua Plácido Prevedello, 77 - Curitiba-PR / zawadzkitrindade@ufpr.br

ABSTRACT. Este trabalho aborda o sequenciamento de produção em uma lavanderia industrial, modelado como um agendamento de máquinas em lote paralelas com famílias incompatíveis. Para solucionar cenários de grande escala, foram desenvolvidas duas variantes da meta-heurística *Adaptive Large Neighborhood Search* (ALNS), sendo a primeira baseada em operadores heurísticos (ALNS 1), e a segunda uma abordagem híbrida utilizando *solver* (ALNS 2). Ambas as versões alcançaram a otimalidade matemática em instâncias de até 50 tarefas. No entanto, em problemas de grande porte, a primeira superou consistentemente a segunda em qualidade e tempo, sendo que o alto volume de iterações rápidas da abordagem clássica mostrou-se mais eficiente do que a busca local exata.

Key-words: Otimização; Sequenciamento em lotes; Meta-heurística ALNS.

1 Introdução

No cenário industrial contemporâneo, a competitividade é definida pela atuação em mercados saturados, nos quais as organizações frequentemente perdem a liberdade de ditar o preço final de seus produtos ou serviços, passando a figurar como tomadoras de preços estabelecidos por variáveis externas. Nesse contexto, a excelência deve ser buscada através da gestão de processos internos, e a rentabilidade torna-se dependente direta da eficiência operacional, seguindo a lógica de que a redução de gastos operacionais é o principal vetor para a maximização de resultados. Consequentemente, o refinamento do uso dos recursos produtivos e a otimização da capacidade instalada deixam de ser meras tarefas administrativas para se tornarem ferramentas de sobrevivência institucional (Pinedo, 2008).

Um setor em que essa necessidade torna-se particularmente crítica é o setor de higienização e lavagem industrial. Diferentemente das lavanderias hospitalares ou hoteleiras, que operam com cargas majoritariamente homogêneas, as unidades voltadas ao atendimento de indústrias e à limpeza de Equipamentos de Proteção Individual (EPIs) lidam com uma vasta heterogeneidade de tecidos e tipos de sujeira. Essa diversidade de materiais e as especificidades técnicas de cada processo impõem um elevado grau de complexidade

ao planejamento produtivo, tornando o sequenciamento uma tarefa desafiadora. Apesar disso, observa-se que nesse ramo a programação da produção ainda é realizada, em sua maioria, de forma empírica, fundamentada na experiência dos operadores e evidenciando uma lacuna na adoção de tecnologias de sequenciamento (ANEL, 2023).

A transição do modelo empírico para o otimizado apresenta restrições técnicas severas, identificadas na literatura especializada como restrições de famílias incompatíveis (Cheng et al., 2014). No ambiente de uma lavanderia industrial, tais restrições manifestam-se na impossibilidade de processar itens e peças de roupas de diferentes categorias distintas em um mesmo ciclo de lavagem, dadas as exigências técnicas e de qualidade (Uzsoy, 1994). Esta necessidade de formação de lotes homogêneos, somada à restrição de capacidade física finita das máquinas de lavagem, eleva consideravelmente a dificuldade de planejamento e torna a tomada de decisão humana propensa a subutilização de recursos (Lee et al., 1992).

Para solucionar esses desafios, a Pesquisa Operacional oferece distintas ferramentas analíticas fundamentadas em modelagem matemática e abordagens heurísticas. Enquanto os métodos exatos buscam a garantia matemática da melhor solução possível por meio da exploração sistemática do problema (Parsa et al., 2010), as heurísticas e meta-heurísticas focam na obtenção de soluções satisfatórias de alto desempenho em tempos computacionais reduzidos e mais eficientes, sendo excelentes alternativas para o suporte à decisão em ambientes dinâmicos (Melouk et al., 2004). Essas abordagens visam substituir a operação empírica por modelos robustos capazes de endereçar a complexidade desses ambientes produtivos. O problema em questão é classificado academicamente como Programação de Máquinas em Lotes (*Batch Processing Machines*), cujo objetivo primordial reside na minimização do tempo total de processamento, ou *makespan*. Estudos relevantes, como os desenvolvidos por Cheng et al. (Cheng et al., 2014), investigam a alocação de famílias incompatíveis em máquinas únicas, ao passo que pesquisas mais recentes, como a de Muter (Muter, 2020), propõem algoritmos avançados para endereçar a complexidade inerente a sistemas de máquinas paralelas operando simultaneamente.

A aplicação prática dos fundamentos discutidos manifesta-se no estudo de caso de uma lavanderia industrial situada em Curitiba, Paraná, inserida em um ambiente de elevada competitividade setorial que conta com mais de dez concorrentes diretos na região. A empresa opera sob uma estrutura de custos na qual a precificação é estabelecida por peso (kg), porém a precificação ao cliente final é associada ao volume de peças individuais higienizadas, o que torna o aproveitamento máximo da capacidade instalada um fator crítico para a rentabilidade do negócio. Atualmente, a programação no formato empírico — ou seja, baseado na experiência dos operadores — resulta em subutilização, perda de capacidade útil e desperdício de insumos produtivos (ANEL, 2023).

Neste trabalho, foi desenvolvido um modelo matemático para solução do problema identificado, em conjunto com uma meta-heurística *Adaptive Large Neighborhood Search* (ALNS) (Ropke, 2006). Dessa forma, integrando o sequenciamento de máquinas e a construção automatizada de lotes de lavagem, buscou-se mitigar o *makespan* operacional e promover eficiência para a empresa em estudo. Os resultados obtidos comprovam a robustez do modelo proposto na geração de soluções factíveis e de alto desempenho, mesmo diante de instâncias de elevada larga escala, apresentando um aproveitamento superior

da capacidade de carga e uma redução consistente no tempo total de processamento.

2 Revisão da literatura

A fundamentação teórica de problemas de agendamento em máquinas de processamento em lotes (*Batch Processing Machines* - BPM) estabelece-se sobre os princípios de Pinedo (2008), que sistematiza as variáveis fundamentais de tarefas, máquinas e objetivos operacionais. O autor define que, em ambientes de BPM, a eficiência é ditada pela formação de lotes em que o tempo de processamento é determinado pela tarefa mais longa do grupo, exigindo uma coordenação precisa entre o tempo de processamento e o tamanho físico dos itens. Sobre essa estrutura, Lee et al. (1992) provaram que o problema de formar lotes com tamanhos diferentes é NP-difícil no sentido estrito (Garey and Johnson, 1979), evidenciando que não existem algoritmos pseudo-polinomiais para resolver tais instâncias.

Na tentativa de solucionar esse desafio matematicamente, Uzsoy (1994) estudou o agendamento de uma máquina única com tarefas de tamanhos arbitrários, e desenvolveu um algoritmo de *Branch and Bound* que obteve sucesso para instâncias de até 20 tarefas, mas demonstrou que heurísticas simples conseguem resultados muito próximos do ótimo com baixo esforço computacional.

Parsa et al. (2010) propuseram um algoritmo de *Branch and Price* baseado em geração de colunas também para o problema de máquina única, e conseguiram resolver problemas de até 100 tarefas. Entretanto, os experimentos realizados demonstraram que a eficiência do algoritmo é altamente sensível à capacidade física da máquina, tendo em vista que a convergência para a solução do método piora drasticamente quando a capacidade da máquina permite o agrupamento de muitas peças pequenas.

Entrando no contexto de máquinas paralelas, Muter (2020) desenvolveu algoritmos de Geração de Colunas e Cortes (*Column-and-Cut*) para minimizar o *makespan*, conseguindo também atingir o ótimo para problemas limitados a 100 tarefas, e ressaltando que instâncias maiores permanecem como um gargalo para modelos puramente matemáticos.

Além disso, Trindade et al. (2018) provaram que, em máquinas paralelas, a simetria entre soluções numericamente diferentes eleva o tempo de processamento de *solvers* convencionais devido à intensa comparação entre resultados praticamente idênticos, e implementaram formulações de programação inteira mista com restrições de quebra de simetria. Dessa forma, reduziram drasticamente o tempo de processamento necessário para alcançar a solução ótima em instâncias de até 100 tarefas.

Por outro lado, Cheng et al. (2014) investigaram o impacto da inclusão de famílias de tarefas incompatíveis ao problema, em que itens de categorias diferentes não podem compartilhar o mesmo lote. Utilizando modelos MIP e heurísticas baseadas em *First Fit* (FF) e *Longest Processing Time* (LPT), os autores provaram que a restrição de famílias nas tarefas reduz a flexibilidade de agrupamento de lotes, tornando os modelos matemáticos mais rígidos e prejudicando o desempenho de *solvers* convencionais à medida que a escala do problema aumenta.

Por fim, Druetto et al. (2023) unificaram todas essas variáveis e observaram um fenômeno importante: embora a incompatibilidade de famílias seja uma restrição produtiva, ela facilita a busca matemática ao reduzir as combinações viáveis de lotes. O estudo comparou o uso de *solvers* exatos com uma heurística de arredondamento, demonstrando que, para 200 tarefas, o *solver* frequentemente atinge o limite de tempo sem encontrar a solução, enquanto a heurística entrega resultados de alta qualidade rapidamente.

3 Desenvolvimento

Este capítulo detalha a engenharia da solução proposta para o agendamento da lavanderia industrial, percorrendo o caminho desde a abstração teórica até a validação prática dos resultados. Inicialmente, apresenta-se a formalização do problema através de um modelo de MIP (3.1), consolidando as restrições fundamentais de capacidade, compatibilidade de famílias e quebra de simetria. Em seguida, descreve-se a estratégia de construção (3.2) utilizada para gerar uma solução inicial factível, que serve de base para a aplicação da meta-heurística ALNS (*Adaptive Large Neighborhood Search*), que é tratada em 3.3 com detalhes sobre seu funcionamento, e operadores de destruição e reconstrução. Por fim, o trabalho valida a eficácia da abordagem desenvolvida através da análise dos resultados, comparando o desempenho da heurística com métodos exatos e uma abordagem híbrida para demonstrar o equilíbrio entre qualidade de solução e eficiência computacional.

3.1 Definição do problema e modelo matemático

O problema abordado nesta pesquisa classifica-se como um agendamento de máquinas de processamento em lote em paralelo (*Parallel Batching Machine Scheduling*) com tarefas de tamanhos não idênticos e famílias incompatíveis. Para formalizar esta classificação, utiliza-se a notação de três campos $\alpha|\beta|\gamma$ consolidada por Pinedo (2008), que descreve, respectivamente, o ambiente de máquinas, as restrições operacionais e a função objetivo. Nessa visão, o modelo é definido como $P_m|p\text{-batch}, s_j, \text{incomp}|C_{max}$, onde:

- P_m : identifica o ambiente de m máquinas paralelas idênticas;
- $p\text{-batch}$: refere-se ao processamento em lotes paralelos;
- s_j : indica a heterogeneidade no tamanho das tarefas;
- incomp : representa a presença de famílias de produtos incompatíveis;
- C_{max} : estabelece o critério de desempenho a ser minimizado.

No contexto da lavanderia industrial, isso significa que as roupas não são processadas individualmente, mas agrupadas em lotes que compartilham o mesmo ciclo de lavagem (receita). A complexidade do problema reside na necessidade de atender a três critérios simultâneos: respeitar a capacidade física de peso de cada máquina (S_m), garantir que itens com processos de lavagem distintos não sejam misturados (famílias incompatíveis) e observar quais máquinas possuem a compatibilidade para cada tipo de lavagem (MF_f). Seguindo essa lógica, o objetivo é minimizar o *makespan* (C_{max}), que representa o ins-

tante de tempo em que a última máquina do sistema encerra sua operação, otimizando assim a vazão total da planta.

Para a construção do modelo matemático para o problema, adotou-se uma formulação baseada em *slots* ou posições de lotes, uma estratégia que facilita a imposição de restrições. A seguir, detalham-se os conjuntos, parâmetros e variáveis de decisão utilizados para estruturar as restrições e a função objetivo deste trabalho.

Conjuntos

- J : Conjunto de todos os produtos (*jobs* ou tarefas) j a serem processados.
- M : Conjunto de máquinas paralelas disponíveis m .
- F : Conjunto de famílias de produtos f .
- K_m : Conjunto de lotes potenciais na máquina m .
- F_j : Índice da família à qual o *job* j pertence.
- MF_f : Subconjunto de máquinas M compatíveis com a família f .

Parâmetros

- s_j : Peso físico (em kg) do produto j .
- S_m : Capacidade máxima de carga da máquina m .
- P_f : Tempo de processamento da família f .

Variáveis

- $x_{jkm} \in \{0,1\}$: 1 se o *job* j for alocado ao lote k da máquina m ; 0 caso contrário.
- $y_{kmf} \in \{0,1\}$: 1 se o lote k da máquina m for designado para a família f ; 0 caso contrário.
- C_{max} : Variável contínua que captura o tempo máximo de operação entre as máquinas (*makespan*).

A partir disso, o modelo é regido pelas seguintes equações:

$$\sum_{m \in M} \sum_{k \in K_m} x_{jkm} = 1 \quad \forall j \in J \quad (1)$$

$$\sum_{j \in J} s_j \cdot x_{jkm} \leq S_m \cdot \left(\sum_{f \in F} y_{kmf} \right) \quad \forall m \in M, k \in K_m \quad (2)$$

$$\sum_{f \in F} y_{kmf} \leq 1 \quad \forall m \in M, k \in K_m \quad (3)$$

$$x_{jkm} \leq y_{km, F_j} \quad \forall j \in J, m \in M, k \in K_m \quad (4)$$

$$C_{max} \geq \sum_{k \in K_m} \sum_{f \in F} P_f \cdot y_{kmf} \quad \forall m \in M \quad (5)$$

$$y_{kmf} = 0 \quad \forall f \in F, m \notin MF_f, k \in K_m \quad (6)$$

$$\sum_{f \in F} y_{kmf} \geq \sum_{f \in F} y_{k+1,m,f} \quad \forall m \in M, k \in \{1, \dots, |K_m| - 1\} \quad (7)$$

A equação 1 garante que cada *job* seja processado exatamente uma vez. Já a equação 2 estabelece o limite físico de carga da máquina, incorporando um interruptor com y_{kmf} : a capacidade S_m só é liberada se o lote k for ativado para alguma família ($\sum y_{kmf}$). A restrição imposta pela equação 3) impede que um lote misture receitas distintas, garantindo o critério monofamília nos agrupamentos. De forma complementar, a equação 4 torna-se essencial para a modelagem, utilizando uma formulação forte para garantir que o *job* j só pode ser alocado ao lote se este lote estiver configurado especificamente para a sua família (F_j).

O *makespan* é definido pela desigualdade em 5, em que o tempo de conclusão de cada máquina deve ser menor ou igual a C_{max} . A equação 6 expressa a construção do modelo já considerando a compatibilidade técnica entre lotes e famílias, zerando e limitando variáveis y_{kmf} de lotes em máquinas não autorizadas para certas famílias.

Como última restrição, a equação 7 aborda a redundância computacional em máquinas paralelas originada pelo problema de simetria do modelo. Ela obriga o preenchimento sequencial dos lotes, impedindo que o *solver* desperdice esforço testando permutações idênticas que não resultam em um *makespan* final diferente.

Por fim, a função objetivo (8) minimiza o *makespan*:

$$\min \quad C_{max} \quad (8)$$

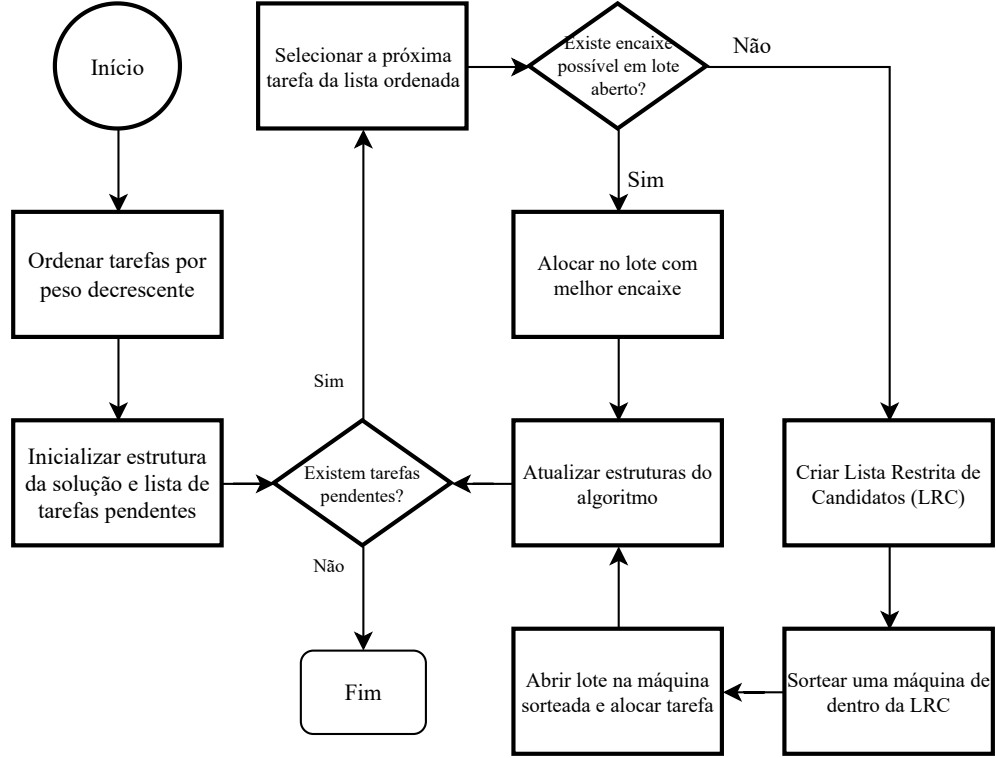
3.2 Heurística de solução inicial

Para iniciar o processo de otimização com meta-heurísticas de busca local, é indispensável construir uma solução inicial que seja, no mínimo, factível perante todas as restrições rígidas do modelo matemático. Algoritmos baseados em buscas em vizinhanças, como o ALNS, apresentam forte dependência da qualidade do ponto de partida. Nesse contexto, soluções iniciais subótimas atrasam a convergência global, enquanto métodos puramente gulosos tendem a gerar soluções idênticas a cada execução, limitando drasticamente a diversidade de exploração do espaço de busca. Dessa forma, neste trabalho foi desenvolvida uma heurística construtiva baseada nos conceitos de aleatoriedade adaptativa do método GRASP (*Greedy Randomized Adaptive Search Procedure*, cuja formalização seminal foi consolidada na literatura por Feo (1995)).

O algoritmo proposto adota uma estratégia de alocação em duas fases para cada tarefa $j \in J$, priorizando a eficiência na ocupação do espaço físico das máquinas antes de expandir

o tempo total de processamento da planta. A dinâmica de funcionamento detalhada da heurística está representada graficamente por meio do fluxograma apresentado na figura 1.

Figura 1: Fluxograma do processo construtivo da solução inicial via GRASP



O processo inicia-se com a ordenação decrescente de todas as tarefas com base em seus respectivos pesos (s_j). Para mitigar o determinismo rígido e permitir a exploração de cenários alternativos, aplica-se uma perturbação estocástica uniforme sobre os pesos, variando no intervalo entre 0.95 e 1.05. A justificativa técnica para ordenar os maiores itens prioritariamente fundamenta-se na teoria clássica dos problemas de empacotamento (*Bin Packing*): tarefas pesadas e volumosas possuem menor flexibilidade de alocação, e postergar sua inserção para o final do processo resultaria em uma quantidade excessiva de lotes subutilizados, fragmentando a capacidade útil das máquinas.

A alocação cumpre regras sequenciais para cada tarefa do conjunto ordenado, dividindo-se em duas etapas principais:

1. **Fase de encaixe guloso:** Para cada tarefa j , o algoritmo realiza uma varredura em todas as máquinas compatíveis com sua família ($m \in MF_f$) e examina os lotes já abertos em cada máquina. Caso encontre lotes ativos que pertençam à mesma

família e que possuam capacidade residual suficiente para suportar a s_j , a tarefa torna-se uma candidata a preencher o lote, e é armazenada computacionalmente. Após passar por todas as máquinas compatíveis, o critério de escolha definitivo adota a política de *Best-Fit*, ou seja, o item é inserido no lote que resultar na menor sobra física de espaço, maximizando a taxa de ocupação do equipamento.

2. **Fase de criação de lotes:** Se nenhum lote ativo puder comportar a tarefa — seja por violação de capacidade de peso ou por incompatibilidade de famílias —, torna-se obrigatório abrir um novo lote no sistema. Para decidir em qual máquina posicionar este novo lote, o algoritmo simula o acréscimo do tempo de processamento da respectiva família (P_f) ao cronômetro atual de operação de cada equipamento compatível, mapeando o impacto incremental no *makespan*. Dessa forma, em vez de selecionar imediatamente a máquina mais vazia, o algoritmo equilibra a qualidade das escolhas e a diversidade das soluções, aplicando a estratégia do método GRASP baseada na construção de uma Lista de Candidatos Restritos (LCR) (Feo, 1995). A LCR funciona como um filtro de desempenho que seleciona apenas as melhores máquinas, descartando aquelas que causariam atrasos excessivos na operação. O rigor desse filtro é controlado pelo parâmetro de aleatoriedade $\alpha = 0.4$.

A LCR estabelece uma nota de corte baseada no custo temporal mínimo ($c_{\min}$) e máximo ($c_{\max}$) projetados entre as máquinas elegíveis, filtrando apenas os equipamentos cujo tempo de conclusão resultante seja inferior ou igual ao limite calculado pela equação 9:

$$Limite = c_{\min} + \alpha \cdot (c_{\max} - c_{\min}) \quad (9)$$

A partir disso, uma máquina é sorteada aleatoriamente dentro deste grupo restrito para receber o novo lote com a respectiva tarefa. Essa aleatoriedade controlada cumpre o papel de diversificar as estruturas de soluções a cada execução do modelo. O ciclo repete-se até que o conjunto de tarefas pendentes seja exaurido, fornecendo uma solução inicial factível para alimentar o algoritmo principal da meta-heurística.

3.3 Algoritmo ALNS

Para superar os limites computacionais dos modelos exatos frente a instâncias de larga escala do problema analisado, este trabalho propõe o emprego da meta-heurística *Adaptive Large Neighborhood Search* (ALNS), introduzida por Ropke (2006). A escolha pelo ALNS justifica-se por sua flexibilidade e resiliência no tratamento de espaços de busca altamente restritos, como o gerado pelo agrupamento das restrições impostas pelo modelo matemático exposto anteriormente. O funcionamento estrutural e o fluxo iterativo do algoritmo implementado seguem a lógica detalhada por meio do pseudocódigo apresentado no Algoritmo 1.

Algorithm 1 Algoritmo ALNS

```
1:  $x = \text{GeraSolInicial}()$ ;  $x^b = x$  ▷ Gera uma solução inicial
2:  $T = T_{max}$ 
3:  $\rho^- = (1, \dots, 1)$ ;  $\rho^+ = (1, \dots, 1)$  ▷ Inicializa os pesos dos operadores igualmente
4: repeat
5:    $d = \text{selecionaD}(\Omega^-, \rho^-)$ 
6:    $r = \text{selecionaR}(\Omega^+, \rho^+)$ 
7:    $x^t = r(d(x, n_{rem}))$  ▷ Destruição e reconstrução solução atual
8:    $\Delta = c(x^t) - c(x)$  ▷ Variação do makespan
9:   if  $\Delta < 0$  then ▷ Aceita solução melhor
10:     $x = x^t$ 
11:   else
12:     Sorteie  $random(0, 1)$ 
13:     if  $random(0, 1) < \exp(-\frac{\Delta}{T})$  then
14:        $x = x^t$  ▷ Aceita solução pior (SA)
15:     end if
16:   end if
17:   if  $c(x^t) < c(x^b)$  then
18:      $x^b = x^t$ 
19:   end if
20:   atualiza  $\rho^-$  e  $\rho^+$  ▷ Atualiza os pesos
21:    $T = \alpha \cdot T$  ▷ Resfriamento da temperatura
22: until Critério de parada
23: return  $x^b$ 
```

O fluxo de execução do ALNS estruturado no algoritmo 1 inicia-se com a parametrização e preparação da busca. Em primeiro momento (linha 1), o algoritmo executa o método de geração da solução inicial descrito na seção 3.2 para construir uma configuração de partida factível, e realiza a calibração da temperatura inicial (T) do sistema (linha 2), estabelecendo o limite máximo (T_{max}) calculado por meio da relação $T_{max} = -0.05 \cdot c(x) / \ln(0.5)$. Na prática, esta expressão garante uma probabilidade inicial de 50% de aceitar soluções que sejam até 5% piores que a solução inicial, permitindo ao algoritmo escapar de vales de ótimos locais nas iterações iniciais.

Para finalizar a etapa de inicialização, são atribuídos valores uniformes e unitários aos vetores de pesos dos operadores de destruição (ρ^-) e de reconstrução (ρ^+), assegurando igual probabilidade de escolha na primeira iteração.

A partir disso, o laço principal do algoritmo segue uma estrutura iterativa até que seja atingido um critério de parada (linha 23), predefinido como um número máximo de iterações.

A cada ciclo do algoritmo, é utilizado o método da roleta para determinar qual será a estratégia de perturbação da iteração (linhas 5 e 6). Esse mecanismo analisa os vetores de pesos atuais (ρ^- e ρ^+) para escolher, de forma proporcional ao desempenho recente do método, um operador de destruição d pertencente ao conjunto Ω^- e um operador de reconstrução r pertencente ao conjunto Ω^+ . Uma vez definidos, a nova solução candidata

(x^t) é construída aplicando sequencialmente os operadores.

Se a solução candidata apresentar uma melhoria no makespan ($\Delta < 0$), esta é incorporada imediatamente como a nova solução atual ($x = x^t$). Por outro lado, caso ocorra uma degradação do valor objetivo, o algoritmo recorre à mecânica probabilística da meta-heurística *Simulated Annealing*, baseada na distribuição de Boltzmann, permitindo que um movimento de piora na solução seja aceito pelo algoritmo e consequente possibilidade de escape de ótimos locais.

De maneira independente a esse processo de aceitação local, as linhas 17 e 18 avaliam a qualidade da solução candidata. Caso o custo de x^t supere o melhor resultado registrado até então ($c(x^t) < c(x^b)$), o algoritmo atualiza a referência permanente do sistema ($x^b = x^t$), guardando o agendamento mais eficiente encontrado ao longo de toda a execução.

Ao final de cada ciclo de iteração, o ALNS realiza a manutenção e atualização de seus parâmetros internos de controle (linhas 20 e 21). Para ajustar os pesos dos operadores utilizados na rodada (ρ^- e ρ^+) com base na qualidade do movimento gerado, o sistema atribui uma pontuação (*score*) baseada na solução encontrada:

- Score = 15: Caso a solução candidata encontre um novo ótimo global ($c(x^t) < c(x^b)$);
- Score = 9: Caso a solução candidata melhore a solução atual, mas não supere a melhor global ($c(x^t) < c(x)$);
- Score = 4: Caso a solução candidata seja pior que a atual, mas seja aceita pelo critério aleatório;
- Score = 0: Caso a solução candidata seja rejeitada pelo sistema.

Com isso, a atualização dos pesos de cada operador ocorre, utilizando um fator de decaimento $\eta = 0.8$, conforme formalizado na equação 10.

$$\rho_{\text{novo}} = \eta \cdot \rho_{\text{antigo}} + (1 - \eta) \cdot \text{score} \quad (10)$$

Por fim, a temperatura do sistema é atualizada linearmente utilizando o fator de decaimento $\alpha = 0.999$ (linha 21). Esse decrescimento progressivo garante que o algoritmo apresente um comportamento exploratório e flexível nas fases iniciais da busca e se comporte de forma refinadora nas iterações finais.

A dinâmica de exploração de vizinhanças do ALNS baseia-se diretamente na aplicação sequencial de operadores de destruição e reconstrução, cujo princípio fundamental, neste trabalho, consiste em perturbar a solução atual por meio da remoção e subsequente reinserção de tarefas. Na primeira etapa, os operadores de destruição extraem um subconjunto de tarefas da estrutura ativa, tornando-as temporariamente livres. Em seguida, os operadores de reconstrução assumem a responsabilidade de realocar esse conjunto de tarefas pendentes de forma factível. O funcionamento de cada um dos operadores implementados neste trabalho está descrito a seguir:

3.3.1 Operadores de destruição (Ω^-)

Antes de determinar quais tarefas serão extraídas, o algoritmo precisa definir o tamanho do impacto dessa remoção. Esse volume de destruição é calculado a cada ciclo por meio do parâmetro n_{rem} , que estipula a quantidade de tarefas a serem retiradas do modelo.

Para definir o valor de n_{rem} , o algoritmo sorteia um coeficiente aleatório contido no intervalo de 0.1 a 0.4, o que corresponde a remover de 10% a 40% do total de tarefas (J). Essa faixa de valores foi estabelecida com base nas recomendações da literatura e reforçada por testes experimentais realizados nesta aplicação, fundamentando-se em critérios de desempenho e convergência da meta-heurística:

- O limite inferior (0.1) garante uma perturbação mínima necessária para o sistema. Caso o algoritmo removesse menos do que 10% das tarefas, a alteração na vizinhança seria sutil demais e faria com que o reconstrutor devolvesse as tarefas para os mesmos lugares e deixasse a busca estagnada em ótimos locais.
- O limite superior (0.4) serve para proteção ao aprendizado do algoritmo. Se a destruição fosse maior que 40%, a estrutura da boa solução construída até o momento seria desfeita, transformando a heurística em um método de reinício aleatório e descartando a otimização das iterações anteriores.

Dessa forma, a variação entre 0.1 e 0.4 assegura que o algoritmo transite entre ciclos de refinamento local e ciclos de grande diversificação. Uma vez definido o valor de n_{rem} da iteração, a escolha de quais tarefas específicas serão removidas da solução atual segue os critérios de um dos quatro operadores desenvolvidos e descritos a seguir:

1. **Destruição da máquina gargalo (Ω_1^-):** Este operador atua diretamente sobre o objetivo de redução do C_{max} da solução, buscando qual máquina possui o maior tempo de processamento na solução atual. Na sequência, o operador extrai lotes inteiros alocados a esse equipamento de maneira aleatória, interrompendo o processo imediatamente assim que o somatório de tarefas contidas nos lotes removidos atinja o limite estabelecido por n_{rem} .
2. **Destruição por família aleatória (Ω_2^-):** Este operador seleciona aleatoriamente uma única família de produtos. O algoritmo então mapeia todos os lotes correspondentes a essa família distribuídos nas máquinas e realiza um embaralhamento. Esses lotes específicos são eliminados consecutivamente até que a cota total de tarefas determinadas por n_{rem} seja preenchida.
3. **Destruição por revezamento de famílias (Ω_3^-):** Inicialmente, todos os lotes na solução atual são segregados e categorizados em listas para cada família de tecidos. O algoritmo então executa ciclos consecutivos de remoção, extraindo um lote de cada família por rodada até que o limite de tarefas definido por n_{rem} seja removido, garantindo que a perturbação ocorra de maneira equilibrada.
4. **Destruição por revezamento de máquinas (Ω_4^-):** Este operador também emprega a lógica de escolha circular, porém orientada aos equipamentos em paralelo.

O algoritmo embaralha os lotes internamente em cada máquina e realiza a remoção de forma alternada, retirando um lote de cada máquina por ciclo, até que o número total de tarefas acumuladas satisfaça o critério do laço principal do ALNS.

3.3.2 Operadores de reconstrução (Ω^+)

Após a etapa de destruição, a lista de tarefas sem alocação gerada pelos destruidores é processada por um dos operadores de reconstrução disponíveis:

1. **Reconstrução gulosa (Ω_1^+):** Este operador reordena as tarefas removidas em ordem decrescente com base em seus respectivos pesos físicos (s_j). Inicialmente, o algoritmo percorre os lotes restantes nas máquinas compatíveis e tenta inserir cada produto utilizando a política *Best-Fit*, que consiste em examinar a capacidade residual disponível em cada lote ativo e selecionar aquele que resultará na menor sobra física de espaço após a inclusão da tarefa, maximizando a taxa de ocupação do lote. Caso o encaixe físico seja infactível em todos os lotes ativos devido à capacidade da máquina (S_m), o operador abre obrigatoriamente um novo lote, posicionando-o na máquina compatível que possuir o menor tempo acumulado de operação até aquele momento.
2. **Reconstrução por *Regret* (Ω_2^+):** Para cada tarefa removida, e antes de realizar qualquer alocação, o algoritmo mapeia e armazena temporariamente os custos de inserção da tarefa em cada uma das máquinas compatíveis e em seus lotes. Para isso, o operador calcula os custos em dois cenários distintos:
 - **Encaixe em lotes ativos:** O algoritmo varre os lotes já abertos em cada máquina compatível, buscando por algum lote ativo que pertença à mesma família da tarefa e possua capacidade disponível. Caso encontrado este cenário, a opção de alocação é armazenada com um custo de -1 . Dessa forma, garante-se prioridade ao preenchimento do espaço físico disponível.
 - **Abertura de novos lotes:** Por outro lado, caso seja necessário abrir um novo lote naquela máquina para alocação da tarefa, o custo dessa alternativa é projetado pelo impacto temporal da abertura, e equivale ao tempo de processamento da família da tarefa (P_f) acrescido ao cronômetro atual da máquina.

Após avaliar essas possibilidades, o operador terá criado um vetor de custos de alocação para cada tarefa disponível, e ordena-o de forma crescente. A partir disso, são registradas a melhor opção de alocação (de custo c_1) e a segunda melhor opção disponível (de custo c_2).

O valor de *regret* é definido pela diferença ($c_2 - c_1$), traduzindo o custo de oportunidade ou prejuízo imediata que o sistema sofrerá caso a tarefa não seja alocada em sua escolha ideal. Um valor elevado de *regret* indica que a segunda alternativa de agendamento (c_2) é consideravelmente pior que a primeira.

A partir disso, o algoritmo seleciona a tarefa que registrou o maior valor de *regret*, realiza a alocação que gerou seu custo ideal (c_1) e remove a tarefa da lista de

pendências. Com isso, a estrutura da solução é atualizada com a alocação e o cronômetro das máquinas atualizado. Como a inserção de uma tarefa altera a ocupação dos lotes e os tempos das máquinas para a próxima iteração, todo o ciclo de simulação de cálculo do *regret* é reiniciado e recalculado para os itens restantes, até que todos estejam devidamente realocados.

3. **Reconstrução exata por modelo matemático (Ω_3^+):** Por fim, este operador integra a programação matemática exata ao laço heurístico, utilizando *solver*. Para viabilizar a execução desse método sem comprometer o tempo computacional do algoritmo, o modelo matemático é construído uma única vez na etapa de configuração inicial, previamente ao início do ALNS, utilizando um limite superior de lotes (K_{max}) calculado para delimitar as matrizes de variáveis de decisão. Quando executado na fase de reconstrução, o operador aplica uma estratégia do tipo *Fix-and-Optimize*, separando o espaço de busca entre variáveis fixas (preservadas da destruição) e variáveis livres (tarefas a serem otimizadas pelo *solver*).

O fluxo do operador inicia-se com o mapeamento dos elementos remanescentes da solução parcial. O algoritmo identifica a posição exata do lote de cada tarefa que sobreviveu à etapa de destruição, registrando quais lotes da fábrica encontram-se parcialmente ocupados, para assim liberar o modelo matemático para edições e iniciar a fixação de limites das variáveis.

Para as tarefas livres, as variáveis binárias de alocação ($x_{j,k,m}$) são totalmente liberadas, assumindo limites entre 0.0 e 1.0. De forma contrária, para cada tarefa fixada na solução parcial, a variável correspondente à sua posição atual de máquina e lote é travada em 1.0, enquanto todas as suas demais combinações de alocações são forçadas a 0.0. Esse mesmo princípio estende-se às variáveis de configuração de lote ($y_{k,m,f}$): se um lote contém tarefas fixas, a variável atrelada à família daquelas tarefas é congelada em 1.0, e as demais famílias para aquele lote são zeradas. Lotes que foram completamente esvaziados ou que já estavam vazios têm suas variáveis $y_{k,m,f}$ liberadas para receber qualquer família compatível.

Com o espaço de soluções reduzido devido ao congelamento da maior parte do cronograma, o operador submete o subproblema ao *solver*, atribuindo alguns parâmetros de desempenho para garantir a alta velocidade exigida pelo ALNS. Após a execução do método e otimização do modelo, caso o *solver* encontre uma configuração viável, o algoritmo varre os valores das variáveis para extrair as decisões ótimas e reconstrói a estrutura da solução, devolvendo o agendamento para o fluxo principal do ALNS.

4 Resultados computacionais

Este capítulo apresenta os resultados computacionais para a resolução do problema de *Parallel Batching Machine Scheduling*. A subseção 4.1 detalha a geração e caracterização das instâncias de teste utilizadas, e na sequência a seção 4.2 expõe de forma consolidada os resultados obtidos, mapeando os limites do modelo exato e comparando o desempenho das variantes heurísticas propostas.

4.1 Geração das instâncias de teste

A validação experimental do algoritmo foi conduzida por meio de um conjunto de 8 instâncias sintéticas construídas de forma aleatória. A construção desses cenários foi estruturada para refletir uma progressão gradual na complexidade do problema, partindo de ambientes altamente controlados até cenários de grande escala industrial. Essa variação dimensional foi planejada para avaliar a escalabilidade da meta-heurística e mapear o limite operacional onde os métodos exatos perdem a capacidade de convergência em tempo hábil.

A modelagem dos parâmetros individuais de cada instância obedeceu a limites de distribuição com base nos seguintes critérios, todos gerados seguindo uma distribuição uniforme:

- **Pesos das tarefas (s_j):** Gerados aleatoriamente no intervalo contínuo entre 0.5 kg e 35.0 kg, simulando a diversidade real de lotes de roupas.
- **Capacidades das máquinas (S_m):** Distribuídas em uma faixa entre 20.0 kg e 200.0 kg para caracterizar um conjunto composto por equipamentos não idênticos.
- **Tempos de processamento (P_f):** Estipulados de forma aleatória entre os limites de 30.0 e 120.0 minutos por ciclo.
- **Compatibilidade técnica (MF_f):** Determinada por amostragem aleatória de máquinas compatíveis.

Para fins de análise estruturada, as instâncias foram segmentadas em três faixas dimensionais distintas: os cenários de pequena escala (15 e 30 tarefas), voltados a testes iniciais; as dimensões de escala real (50, 70 e 100 tarefas), que refletem o fluxo cotidiano apontado pela lavanderia do estudo de caso; e as configurações de grande escala (150, 300 e 500 tarefas), incorporadas com o intuito de estressar o sistema e testar os limites do algoritmo. A Tabela 1 consolida a estrutura final de cada cenário gerado.

Tabela 1: Dimensões estruturais das instâncias de teste geradas para o agendamento em lotes

Instância	Tarefas ($ J $)	Máquinas ($ M $)	Famílias ($ F $)
Instância 1	15	4	3
Instância 2	30	5	7
Instância 3	50	7	10
Instância 4	70	5	8
Instância 5	100	8	10
Instância 6	150	10	15
Instância 7	300	12	15
Instância 8	500	15	10

4.2 Resultados obtidos

Inicialmente, como etapa preliminar de *benchmarking* para avaliar os limites da programação exata, tentou-se resolver as instâncias do problema aplicando diretamente o modelo matemático no *solver*. Os testes práticos demonstraram que o *solver* exato conseguiu encontrar e provar a solução ótima global dentro de um tempo limite tolerável (estabelecido em até 30 minutos de execução) apenas para as instâncias de pequena escala, compreendendo os cenários de até 50 tarefas.

A partir da instância de 70 tarefas, a quantidade de restrições e variáveis binárias de alocação expandiram o espaço de busca consideravelmente. Esse fenômeno fez com que o *solver* não conseguisse computar um *gap* de otimalidade aceitável em tempo hábil, reforçando a classificação do problema como NP-difícil e justificando a necessidade do emprego de métodos heurísticos ou mistos para este problema.

Ao partir para a aplicação da abordagem heurística, para garantir a reprodutibilidade dos testes computacionais e estabelecer uma base de comparação justa entre as diferentes configurações, todos os ensaios foram definidos por um conjunto de parâmetros para o algoritmo. Esses valores foram calibrados e ajustados por meio de testes empíricos de sensibilidade aplicados ao cenário de agendamento de lotes. A Tabela 2 consolida todos os parâmetros operacionais integrados ao código desenvolvido.

Tabela 2: Parâmetros de controle calibrados para os algoritmos ALNS

Parâmetro	Valor atribuído
Número máximo de iterações	10000
Tempo limite de execução (<i>max_time</i>)	120.0 s
Fator de ajuste dos pesos (η)	0.8
Coefficiente de resfriamento do Simulated Annealing (α)	0.999
Fator de filtragem do GRASP (α_{grasp})	0.4
Percentual mínimo de destruição	0.10 (10%)
Percentual máximo de destruição	0.40 (40%)
Score para novo ótimo global	15
Score para solução aceita com melhora	9
Score para solução pior aceita pelo SA	4
Score para solução rejeitada	0

Diante da limitação dos métodos puramente exatos exposta anteriormente para resolver cenários de maior magnitude, este trabalho propõe e avalia duas variantes distintas da meta-heurística adaptativa para contornar a explosão combinatória do problema. A primeira configuração, denominada **ALNS 1**, constitui a versão do algoritmo que utiliza estritamente os dois primeiros operadores de reconstrução expostos anteriormente neste trabalho, ou seja, com o uso de heurísticas construtivas (Ω_1^+ e Ω_2^+) para reconstruir as soluções. Por sua vez, a segunda abordagem, denominada **ALNS 2**, caracteriza-se como um arranjo híbrido, estendendo a busca local por meio da incorporação do operador exato por modelo matemático (Ω_3^+). O algoritmo como um todo foi desenvolvido utilizando a linguagem de programação Python, sendo incorporado o pacote *solver* SCIP para

aplicação do método exato.

Para avaliar a estabilidade, a velocidade de convergência e a robustez do algoritmo, ambas as versões testadas (ALNS 1 e 2) foram executadas múltiplas vezes, sendo submetidas a 20 execuções independentes para cada uma das oito instâncias geradas. A Tabela 3 apresenta o compilado consolidado a partir das amostras computacionais coletadas para a primeira versão configurada — **ALNS 1**.

Tabela 3: Desempenho consolidado do ALNS 1

Instância ($J/M/F$)	Makespan Solver	Makespan inicial médio	Makespan final médio	Melhoria média (%)	Pior Makespan	Melhor Makespan	Tempo médio de execução (s)
15/4/3	60.00	80.25	60.00	24,67%	60.00	60.00	0.87
30/5/7	165.00	191.50	165.00	13,44%	165.00	165.00	1.45
50/7/10	178.00	234.30	178.00	23,98%	178.00	178.00	2.46
70/5/8	N/E	302.70	250.00	17,21%	250.00	250.00	4.70
100/8/10	N/E	211.60	176.55	16,04%	177.00	176.00	5.69
150/10/15	N/E	326.75	260.00	20,01%	260.00	260.00	8.45
300/12/15	N/E	451.05	370.85	17,57%	378.00	361.00	25.15
500/15/10	N/E	500.75	451.75	9,68%	455.00	445.00	93.75

Observando a tabela, os resultados confirmam a alta assertividade do algoritmo. Nas instâncias em que o *solver* obteve convergência ($J \leq 50$), a meta-heurística alcançou a otimalidade exata em todas as execuções. Essa estabilidade estende-se até o cenário de 150 tarefas, validando a eficiência dos operadores (Ω^- e Ω^+) e critérios de aceitação estabelecidos para escapar de ótimos locais. Nas instâncias de grande escala ($J \geq 300$), a explosão combinatória inerente à natureza do problema gera dispersões entre as execuções, comportamento esperado diante do limite fixo de iterações do algoritmo.

Quanto à eficiência computacional, o ALNS 1 demonstrou viabilidade prática para tomada de decisão em tempo real no ambiente organizacional. Cenários com escala de até 100 tarefas foram resolvidos em menos de 6.00 segundos, enquanto a instância limite de 500 jobs demandou um tempo médio de apenas 93.75 segundos, respeitando o teto estabelecido de 120.0 segundos.

Uma vez mapeado o comportamento dessa abordagem, os ensaios avançam para a análise da variante ALNS 2. Este algoritmo, por sua vez, foi estruturado sob uma parametrização diferente do ALNS 1 para equilibrar o ganho de qualidade com o esforço computacional da meta-heurística. Fixou-se um limite de convergência local de 5.00 segundos por chamada do operador Ω_3^+ , enquanto o algoritmo global foi restrito a 1000 iterações ou 300.00 segundos de execução. A seguir, são apresentados os resultados consolidados desta segunda configuração.

Tabela 4: Desempenho consolidado do ALNS 2

Instância ($J/M/F$)	Makespan Solver	Makespan inicial médio	Makespan final médio	Melhoria média (%)	Pior Makespan	Melhor Makespan	Tempo médio de execução (s)
15/4/3	60.00	79.50	60.00	24,00%	60.00	60.00	10.02
30/5/7	165.00	201.75	165.00	17,66%	165.00	165.00	142.20
50/7/10	178.00	239.35	178.00	25,58%	178.00	178.00	300.00
70/5/8	N/E	302.00	252.50	16,11%	260.00	250.00	300.00
100/8/10	N/E	204.45	177.70	12,84%	181.00	177.00	300.00
150/10/15	N/E	319.50	265.50	16,70%	280.00	260.00	300.00
300/12/15	N/E	451.25	378.35	15,93%	388.00	372.00	300.00
500/15/10	N/E	496.50	455.00	8,26%	460.00	445.00	300.00

Os resultados demonstram que a variante híbrida manteve a otimalidade exata nas instâncias controladas ($J \leq 50$), novamente igualando-se ao *solver*. Contudo, a inclusão da modelagem matemática impôs o gargalo de que, a partir da instância de 50 tarefas, o tempo médio de execução passou a ultrapassar o limite de 300.00 segundos. Como consequência, o critério de parada por tempo foi atingido em todos os cenários de médio e grande porte, interrompendo a execução antes que o limite de 1000 iterações pudesse ser alcançado, reduzindo a capacidade de exploração do algoritmo. Mesmo assim, a busca local realizada pelo *solver* mostrou-se eficaz, gerando soluções finais robustas.

Para consolidar as vantagens e desvantagens de cada configuração, a seguir é exposta uma tabela comparativa entre os resultados obtidos pelo ALNS 1 e pelo ALNS 2.

Tabela 5: ALNS 1 *vs.* ALNS 2

Instância (<i>J/M/F</i>)	Makespan Solver	ALNS 1			ALNS 2		
		Médio Final	Melhor	Tempo (s)	Médio Final	Melhor	Tempo (s)
15/4/3	60.00	60.00	60.00	0.87	60.00	60.00	10.02
30/5/7	165.00	165.00	165.00	1.45	165.00	165.00	142.20
50/7/10	178.00	178.00	178.00	2.46	178.00	178.00	300.00
70/5/8	N/E	250.00	250.00	4.70	252.50	250.00	300.00
100/8/10	N/E	176.55	176.00	5.69	177.70	177.00	300.00
150/10/15	N/E	260.00	260.00	8.45	265.50	260.00	300.00
300/12/15	N/E	370.85	361.00	25.15	378.35	372.00	300.00
500/15/10	N/E	451.75	445.00	93.75	455.00	445.00	300.00

Nas instâncias de pequeno porte, ambos os algoritmos garantem a otimalidade exata provada pelo *solver*. No entanto, o ALNS 1 atinge esses resultados em frações de segundo, enquanto o ALNS 2 sofre a penalidade temporal devido ao tempo de carregamento e execução do *solver* SCIP a cada iteração. Já nas instâncias de médio e grande porte, em que o modelo exato puro já não converge, o ALNS 1 superou consistentemente o ALNS 2, tanto em qualidade de solução quanto em eficiência computacional.

5 Conclusão

Este trabalho apresentou um modelo de Pesquisa Operacional para resolver o problema de sequenciamento de produção em uma lavanderia industrial. O cenário foi modelado

como um agendamento de máquinas em lote paralelas com famílias incompatíveis e tarefas de tamanhos diferentes ($P_m|p\text{-batch}, s_j, \text{incomp}|C_{max}$). A formulação matemática proposta representou com precisão as restrições práticas da fábrica, e as duas variantes da meta-heurística ALNS desenvolvidas (ALNS 1 e ALNS 2) funcionaram como alternativas eficientes para lidar com a alta complexidade computacional do problema.

Os testes computacionais mostraram que a variante híbrida ALNS 2, que utiliza o operador integrado ao *solver* SCIP, gerou um gargalo no tempo de processamento. O *solver* consumiu muito tempo para resolver os subproblemas locais, fazendo com que o algoritmo atingisse o limite de 300.00 segundos com poucas iterações completas. Por outro lado, a versão puramente heurística ALNS 1 foi mais eficiente. Ao executar operadores clássicos de forma rápida ao longo de um número mais expressivo de iterações, o ALNS 1 explorou melhor o espaço de busca e encontrou soluções com menor *makespan* em tempos significativamente mais baixos.

Do ponto de vista prático, os resultados mostram que o ALNS 1 é uma ferramenta viável para o planejamento da empresa do estudo de caso deste trabalho. O algoritmo alcançou o mesmo resultado ótimo do *solver* nas instâncias menores e manteve um desempenho estável até o cenário com 150 tarefas. Mesmo na maior instância testada, de 500 tarefas, o ALNS 1 resolveu o problema em uma média de apenas 93.75 segundos, ficando abaixo do limite de tempo definido originalmente como parâmetro de execução. Esse desempenho indica que o algoritmo pode substituir o sequenciamento empírico atualmente adotado na lavanderia, permitindo programar a produção de forma automatizada, aproveitar melhor a capacidade das máquinas e refletir diretamente na eficiência do uso de recursos produtivos.

Referências

- ANEL (2023). Perfil do setor de lavanderias no brasil. Technical report, ANEL/SEBRAE, São Paulo.
- Cheng, B., Cai, J., Yang, S., and Hu, X. (2014). Algorithms for scheduling incompatible job families on single batching machine with limited capacity. *Computers & Industrial Engineering*, 75:116–120.
- Druetto, A., Pastore, E., and Renner, E. (2023). Parallel batching with multi-size jobs and incompatible job families. *TOP*, 31(2):440–458. Published online 2022.
- Feo, Thomas A. e Resende, M. G. C. (1995). Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6(2):109–133.
- Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, San Francisco.
- Lee, C.-Y., Uzsoy, R., and Martin-Vega, L. A. (1992). Efficient algorithms for scheduling semiconductor burn-in operations. *Operations Research*, 40(4):764–775.

- Melouk, S., Damodaran, P., and Chang, P.-C. (2004). Minimizing makespan for single machine batch processing with non-identical job sizes using simulated annealing. *International Journal of Production Economics*, 87(2):141–147.
- Muter, İ. (2020). Exact algorithms to minimize makespan on single and parallel batch processing machines. *European Journal of Operational Research*, 285(2):470–483.
- Parsa, N., Karimi, B., and Kashan, A. (2010). A branch and price algorithm to minimize makespan on a single batch processing machine with non-identical job sizes. *Computers & Operations Research*, 37(10):1720–1730.
- Pinedo, M. L. (2008). *Scheduling: Theory, Algorithms, and Systems*. Springer Science & Business Media, New York.
- Ropke, Stefan e Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4):455–472.
- Trindade, R., de Araújo, O., Fampa, M., and Müller, F. (2018). Modelling and symmetry breaking in scheduling problems on batch processing machines. *International Journal of Production Research*, 56(22):7031–7048.
- Uzsoy, R. (1994). Scheduling a single batch processing machine with non-identical job sizes. *International Journal of Production Research*, 32(7):1615–1635.